VIETNAM NATIONAL UNIVERSITY HO CHI MINH CITY
UNIVERSITY OF SCIENCE



WEEKLY REPORT – WEEK 4

Project: "Design Space Exploration of a Shared L2 Cache with gem5"

Student: Luong Khanh Ngoc

ID: 24200134

Week: 29/10 – 02/11/2026

Ho Chi Minh City, November 2026

1. Status This Week

Previous weeks used self-made (synthetic) workloads. Following the mentor's advice, this week moved to **real workloads** that already exist in the CVA6 repository. Goal: later run the same program on both the CVA6 RTL and the gem5 model and compare them.

- **Done:** checked the CVA6 repo and its cache configuration → `cva6_environment.md`
- **Done:** listed every available benchmark and test → `available_tests.csv`
- **Done:** read the code and picked 9 candidate workloads → `workload_list.csv`
- **Done:** proposed data sizes for each workload (22 configurations) → `workload_config.csv`
- **In progress:** trial build and run → vvadd builds, Spike is installed, a run-time error is being fixed
- **Not started:** memory characterization from traces → `characterization.csv` (the program must run first)

2. Terms Used in This Report

- **Workload**: a program run on the CPU to produce the memory traffic being studied.
- **Footprint / array bytes**: total bytes of data the program reads/writes inside the measured region.
- **Line / set / way**: a cache is split into **sets**; each set has several **ways**; each way holds one **line** (a data block). The group's L2: 1 MiB, 16 ways, 64 B lines → 1024 sets, **each way = 64 KiB**.
- **Lines per set**: size ÷ 64 KiB. E.g. 768 KiB → 12 lines per set → at least 12 ways are needed to fit.
- **ROI**: the measured part of the program. In riscv-tests it lies between `setStats(1)` and `setStats(0)`.
- **Warm-up**: running one pass first so the data is already in the cache when measurement starts.
- **Bare-metal**: a program that runs directly on the CPU, without an operating system.
- **Spike**: the RISC-V instruction-set simulator; fast, used to check that a program is correct and to collect address traces.

3. CVA6 Environment

- Versions: CVA6 commit `742dc82` (23/10/2026); riscv-tests commit `f92842f` (the version CVA6 uses). Original sources untouched.
- CPU configuration used: `cv64a6_imafdc_sv39` (64-bit CPU, 1 of 16 available). Values read directly from the configuration file:
  - **L1I** (instruction cache): 16 KiB, 4 ways, 16 B lines.
  - **L1D** (data cache): 32 KiB, 8 ways, 16 B lines, **write-through** (every store is also sent out of the L1).
  - **L2: none** in the simulation model (testharness). Data goes straight: CPU → AXI bus → delay block (axi_delayer) → SRAM memory.
- Tools: RISC-V compilers for bare-metal (GCC 13.2) and Linux (GCC 13.3), Python, Perl; Spike 1.1.1 installed; Verilator (RTL simulator) not yet installed.

4. Benchmarks and Tests Available in CVA6

Everything that runs on CVA6 was grouped into 24 entries. Most are instruction checks, not workloads:

Group	Count	Examples	Workload?
riscv-tests benchmarks	12	vvadd, qsort, spmv, mm, median...	Yes (except pmp)
CVA6 own benchmarks	2	CoreMark, Dhrystone	Yes

Group	Count	Examples	Workload?
Custom functional tests	25 folders	CSR, pmp, sv39, hello_world...	No
ISA / compliance testlists	31 lists	1–220 tests per list	No
Random tests, Linux boot	2	riscv-dv, linux.sh	Not now

Keeping only benchmarks with meaningful memory traffic leaves **9 candidates** (section 5).

5. Selected Workloads

Each workload's access pattern was confirmed by reading its main loop. Size is the total of the arrays used in the measured region, at the original riscv-tests size.

Workload	What it does	Access pattern	Original size	Role
vvadd	add two vectors	sequential (Stream)	3,6 KB	main
median	3-point median filter	sequential, reuses neighbours	3,2 KB	main
qsort	quicksort	data-dependent, shrinking range	8,6 KB	main
rsort	radix sort	scan + scattered writes to 256 buckets	18 KB	main
spmv	sparse matrix × vector	indirect `x[idx[k]]` (Graph-like)	38 KB	main
mm	blocked matrix multiply	same data reused 8× (Loop)	28 KB	main
mt-matmul	simple matrix multiply	column reads, fixed stride	3 KB	vs mm
mt-vvadd	like vvadd (double)	Stream	24 KB	backup
coremark	mixed benchmark	linked list + small matrix	2 KB	reference

> **Key finding:** at the original size every workload is only about 2–39 KB and almost fits in the 32 KiB L1D. Run as-is, the 1 MiB L2 would see almost no traffic and changing the number of ways would show no difference. The **input data must be enlarged** (code unchanged).

Details of the main workloads

- **vvadd** – `c[i] = a[i] + b[i]`: reads 2 arrays, writes 1, step of 4 B. No reuse within a pass → Stream-like. Important for seeing how the L2 handles large data streams (vector/DSP style) and write traffic.
- **median** – reads `input[i-1]`, `input[i]`, `input[i+1]`, writes `results[i]`: sequential, but each element is read 3 times in consecutive iterations → very short reuse, mostly kept by the L1. Represents DSP-style filters.
- **qsort** – partitions by value and swaps (`SWAP`), working on ever smaller sub-ranges, with `stack[50]` remembering pending ranges. Access order depends on the data; small sub-ranges fit in the cache → temporal reuse (Temporal-like).
- **rsort** – sorts 8 bits at a time (4 passes for 32-bit numbers): each pass counts into 256 `buckets[]`, then scatters into `scratch`. Sequential reads but writes to 256 places at once → tests how the L2 copes with many write streams.
- **spmv** – for each row: reads `val[k]`, `idx[k]` sequentially, then `x[idx[k]]` by index → irregular jumps (Graph-like). Already slightly over the L1D at its original size (38 KB), so the best candidate to see the L2 at work.
- **mm** – blocked matrix multiply, repeated 8 times on the same data (`R = 8`); before computing, data is copied into a reordered layout (repack). Strong reuse → Loop-like. Data lives on the stack, so enlarging it needs a code change.
- **coremark** – mixed benchmark: linked-list traversal, small matrix, state machine. Fixed 2000 B of data, so kept as a reference workload, not for L2 tests.

How each workload marks its ROI

- **setStats(1) / setStats(0)** around the kernel: vvadd, median, qsort, rsort, spmv (CoreMark also uses it via `start_time` / `stop_time`).

- **“stats(...)” macro** reading the cycle and instruction counters: mt-matmul, mt-vvadd.
- **Direct counter reads** inside the 8-iteration loop: mm.
- When comparing gem5 with the RTL, the same ROI must be measured on both sides.
- Compared with the earlier synthetic types: Stream ↔ vvadd; Loop ↔ mm, mt-matmul; Temporal ↔ qsort; Graph ↔ spmv.

6. Proposed Data Sizes

Chosen the same way as loop768K: based on **lines per set**. Each workload has up to 3 levels:

- **orig**: original size, used for comparison with the CVA6 RTL.
- **l2fit**: about 768 KiB = 12 lines per set → fits the L2 with all ways on, runs short below 12 ways.
- **l2over**: about 2.25 MiB → larger than the whole L2.

Workload	How to enlarge	l2fit	l2over
vvadd	existing data script	65,536 elements	196,608 elements
median	existing script	98.304	294.912
qsort	existing script	196.608	589.824
rsort	borrow qsort's script	98.304	294.912
spmv	write a new script (seeded)	8192 × 8192, ~51k non-zeros	24576 × 24576
mm	edit size constants in code	128 × 125 × 126	–
mt-matmul	existing script (parameter = matrix side)	256 × 256	448 × 448
coremark, mt-vvadd	unchanged	–	–

22 configurations in total, currently proposed; confirmed after generating data and getting a PASS.

How the sizes are computed

- Size = number of elements × element size × arrays used in the ROI. E.g. vvadd: 65,536 × 4 B × 3 arrays = 786,432 B = **768 KiB** → 768 ÷ 64 = **12 lines per set**.
- median: 98,304 × 4 B × 2 arrays = 768 KiB. qsort: 196,608 × 4 B × 1 array = 768 KiB. rsort: 98,304 × 4 B × 2 arrays (+2 KiB buckets) ≈ 770 KiB.
- spmv: each non-zero costs 12 B (8 B value + 4 B index), plus vectors x, y, ptr → 8192 rows, ~51k non-zeros ≈ 760 KiB.
- Lines per set here assume evenly spread data; the real count per set will be measured from traces.

Groups by how data is generated

- **Group A – script exists** (`--size`, `--seed`): vvadd, median, qsort, mt-vvadd, mt-matmul. Only a parameter change.
- **Group B – borrowed script:** rsort uses qsort's script (positive numbers, same sort order) → a PASS run is needed to confirm.
- **Group C – own work needed:** spmv (original script in Scala, no seed), mm (edit size constants, move arrays off the stack), coremark (kept as is).

Expected L2 behaviour (not yet measured)

- **Stream-like (vvadd, median):** the L2 only matters with a warm-up pass; at l2fit a cliff near 12 ways is expected, like loop768K; at l2over almost every access misses.
- **Loop-like (mm, mt-matmul):** data reused many times → sensitive to the number of ways, a clear cliff is expected once the L2 is smaller than the data.
- **Indirect / data-dependent (spmv, qsort, rsort):** misses are expected to rise gradually as ways are removed, without a sharp cliff, like the earlier Graph and Temporal.

- These expectations come from the synthetic workloads; they will be checked against real traces.

7. Trial Build and Run

Building vvadd (original)

- First attempt failed with `string.h: No such file`: the Ubuntu compiler ships without a C library.
- Fix: use the installed **picolibc** library by adding `--specs=picolibc.specs`.
- Second attempt **succeeded**: `vvadd.riscv` is 21,304 B, of which 3,672 B is data.
- 3,672 B matches the hand calculation (3 arrays × 300 numbers × 4 B = 3,600 B + 72 B other) → the first size confirmed on a real file.
- File check: 64-bit program starting at `0x80000000`, reporting completion through the `tohost` memory cell → the bare-metal form Spike can run.

Running on Spike

- Spike 1.1.1 is installed (built from upstream source; the first build missed one sub-library, now fixed).
- Running vvadd: the computation finishes, but the program **stops with an error while printing the result** (Spike reports a trap, code 1337). The faulting instruction is being located; picolibc is suspected.
- Good news: Spike's log **records the address of every memory read/write** (`mem 0x…`), so traces for memory characterization can be collected.

8. Important Notes

- **Original data is too small** for the caches (2–39 KB vs. 32 KiB L1D, 1 MiB L2) → it must be enlarged to study the L2.
- **Different build flags:** CVA6's script uses `-O3` with no warm-up; the riscv-tests Makefile uses `-O2` with `PREALLOCATE`. The orig configs keep CVA6's flags for RTL comparison.
- **PREALLOCATE does not always warm the cache:** vvadd, median, spmv run one pass on the same data first → warm; qsort, rsort run on a different array → measured data stays cold.
- **CVA6 L1 lines are only 16 B**, while the planned L2 uses 64 B → 1 L2 line = 4 L1 lines. gem5 uses one line size for the whole system, so this is not yet modelled exactly.
- **L1D is write-through:** every store goes out to the L2, so the L2 sees all write traffic.
- **Bare-metal programs do not run directly in gem5 SE mode** (which needs Linux programs) → a Linux build or gem5's bare-metal mode is needed.
- **spmv:** the original script has no fixed seed, so each run produces different data → the new script must use a seed for repeatable results.
- **Array bytes are not the real working set:** traces are needed to see what the cache actually sees.